

1. PORTADA

EVALUACIÓN CONTACTO CON EL DOCENTE

MATERIA:

ESTRUCTURA, MODELADO, ALMACENAMIENTO DE BDD

DOCENTE:

ROSERO AGUAS MIGUEL ÁNGEL

FACULTAD:

INGENIERÍA

CARRERA:

INGENIERÍA EN SOFTWARE

FECHA DE ENTREGA:

28/06/2026

Tabla de contenido

2. ENTREGABLE DE REQUISITOS.....	3
3. HERRAMIENTAS UTILIZADAS	4
4. EVIDENCIA TÉCNICA.....	5
4.1. Modelo lógico de la base de datos.	5
4.2. Código de Python.	5
4.3. Menú del sistema.	6
4.4. Evidencia de funcionamiento.	6
5. AUTOEVALUACIÓN.....	7
REFERENCIAS.....	8

2. ENTREGABLE DE REQUISITOS

Se desarrolló una aplicación de consola en Python, la cual está conectada a una base de datos relacional en SQL Server, orientada a gestionar una plataforma de streaming educativo. A diferencia de las plataformas de entretenimiento convencionales, este sistema se enfoca en contenido académico y cultural, incluyendo documentales, tutoriales, podcasts y material científico, también permite administrar usuarios, suscripciones, contenido educativo, episodios, historial de visualizaciones y favoritos.

Los objetivos del sistema son:

- Gestionar el registro y la administración de usuarios con sus datos personales.
- Controlar las suscripciones por plan: Individual (1 cuenta, 2 dispositivos), Dúo (2 cuentas, 5 dispositivos) y Familiar (5 cuentas, 14 dispositivos).
- Almacenar y consultar contenido educativo clasificado por tipo (Documental, Tutoria, Podcasts) y categoría (Ciencia, Historia, Arte, Matemáticas, etc.).
- Registrar el historial de visualización de cada usuario, incluyendo progreso y estado de completado.
- Permite operaciones CRUD completas desde una interfaz de consola interactiva.
- Ejecutar consultas avanzadas que permitan analizar el comportamiento de los usuarios dentro de la plataforma.

3. HERRAMIENTAS UTILIZADAS

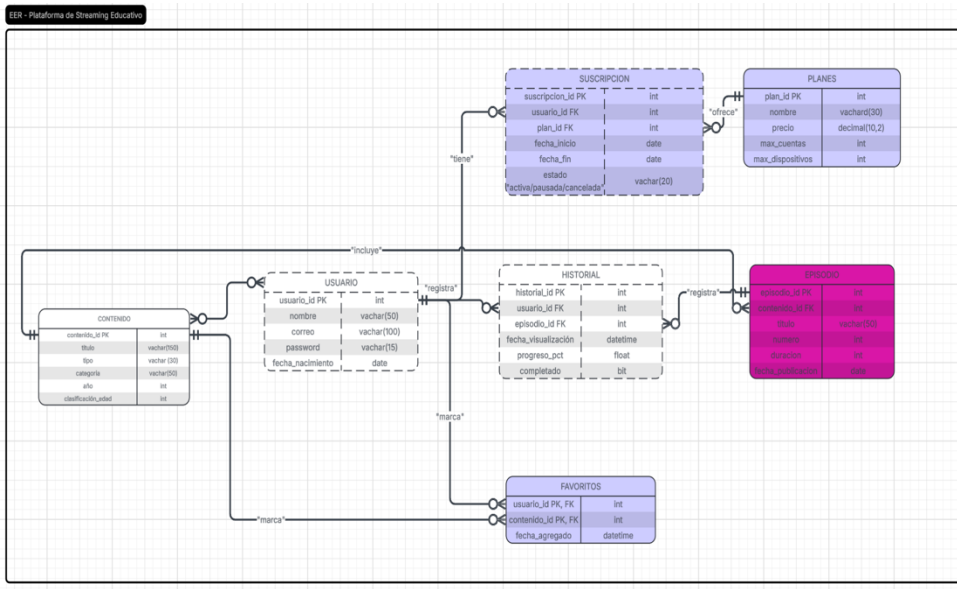
Para desarrollar e implementar los sistemas utilicé las siguientes herramientas:

- **Python:** Fue el lenguaje de programación elegido para desarrollar la aplicación de consola. Se empleó la librería **pyodbc** para establecer la conexión con la base de datos.
- **Visual Studio Code:** Entorno de desarrollo integrado (IDE), fue utilizado para escribir y ejecutar el código Python, así como para gestionar las consultas de SQL mediante la extensión MSSQL
- **Microsoft SQL Server 2022:** Sistema Gestor de Bases de Datos (SGBD) utilizado para almacenar y administrar la información de la plataforma.
- **Docker Desktop:** Debido a que poseo el sistema operativo macOS y quería usar SQL Server, se ejecutó dentro de un contenedor Docker, ya que no existe una versión nativa de SQL Server para Mac.
- **Microsoft ODBC Driver 18 for SQL Server:** Driver de conexión instalado en macOS para permitir la conexión entre Python y SQL Server a través de pyodbc.

Todas estas herramientas fueron integradas para lograr un entorno de desarrollo funcional en macOS, permitiendo simular un ambiente similar al de Windows donde SQL Server opera de forma nativa.

4. EVIDENCIA TÉCNICA.

4.1. Modelo lógico de la base de datos.



La figura muestra el modelo lógico de la base de datos utilizada para el sistema Streaming Educativo. Se observan las relaciones entre las tablas usuario, planes, contenido, episodio, suscripción, historial

4.2. Código de Python.

Se estableció la conexión entre Python y SQL Server mediante la librería, utilizando el Driver ODBC 18 para SQL Server.

```
1 import pyodbc
2
3 # Conexión a la base de datos
4 conn = pyodbc.connect(
5     'DRIVER={ODBC Driver 18 for SQL Server};'
6     'SERVER=localhost,1433;'
7     'DATABASE=STREAMING_EDUCATIVO;'
8     'UID=sa;'
9     'PWD=TuPassword123!;'
10    'TrustServerCertificate=yes;'
11 )
```

```
64 cursor.execute("INSERT INTO USUARIO (nombre, correo, password, fecha_nacimiento) VALUES (?, ?, ?, ?)",
65                nombre, correo, password, fecha)
```

Esta sentencia permite insertar un nuevo usuario utilizando consultas parametrizadas para evitar errores y facilitar el ingreso de datos.

```
105 try:
106     cursor.execute("DELETE FROM USUARIO WHERE usuario_id = ?", id)
107     conn.commit()
108     print("✅ Usuario eliminado!")
109 except Exception as e:
110     print(f"❌ No se puede eliminar: tiene registros relacionados.")
111     conn.rollback()
```

Esta consulta identifica a los tres usuarios que poseen la mayor cantidad de contenidos marcados como favoritos.

Se agregó el manejo de excepciones para controlar errores de integridad referencial cuando un registro tiene relaciones con otras tablas.

```
162 def consultas_avanzadas():
163     print("\n===== CONSULTAS AVANZADAS =====")
164
165     print("\n1. Top 3 usuarios con más contenido en favoritos:")
166     cursor.execute("""
167         SELECT TOP 3 U.nombre, COUNT(F.contenido_id) AS total_favoritos
168         FROM USUARIO U JOIN FAVORITOS F ON U.usuario_id = F.usuario_id
169         GROUP BY U.nombre
170         ORDER BY total_favoritos DESC;
171     """)
```

4.3. Menú del sistema.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS Python

```
/opt/homebrew/bin/python3 /Users/nathalymirleyperaltamendoza/streaming_app.py
○ nathalymirleyperaltamendoza@MacBook-Pro-de-Nathaly / % /opt/homebrew/bin/python3 /Users/nathalymirleyperaltamendoza/streaming_app.py

===== STREAMING EDUCATIVO =====
1. Ver todas las tablas
2. Insertar registro
3. Eliminar registro
4. Actualizar registro
5. Consultas avanzadas
6. Salir
Elige una opción: []
```

4.4. Evidencia de funcionamiento.

Elige una opción: 2

¿Qué deseas insertar?

1. Usuario

2. Contenido

Elige: 1

— Usuarios actuales —

(1, 'Emely', 'emely1104pm@gmail.com', '13579', datetime.date(2003, 4, 11))

(2, 'Ingrid', 'ingridmloor86@gmail.com', '24680', datetime.date(1986, 1, 20))

(3, 'Enrique', 'galopsolorzano79@gmail.com', '12358', datetime.date(1979, 7, 19))

(4, 'Geovanny', 'peraltageovanny7@gmail.com', '837478', datetime.date(2012, 8, 20))

(5, 'Teresita', 'teresitaloor1986@gmail.com', '461876', datetime.date(1997, 2, 4))

(6, 'Nathaly', 'peraltanathaly77@gmail.com', 'mdh682b', datetime.date(2007, 5, 26))

(7, 'Luis', 'mendozaluis73@gmail.com', '32784h783', datetime.date(2004, 5, 17))

(8, 'Mirley', 'mirelymendoza7@gmail.com', '9876n27', datetime.date(2007, 7, 7))

(9, 'Iris', 'ristak753idk@gmail.com', 'pdt#heni98', datetime.date(2000, 7, 1))

(10, 'Shady', 'shaidyyygirl@gmail.com', 'DaKa#37idc', datetime.date(2008, 11, 28))

Nombre: Keyla

Correo: keynotarelive87@gmail.com

Password: jwfhe8375n

Fecha nacimiento (YYYY-MM-DD): 1977-05-07

✔ Usuario insertado!

— Usuarios actualizados —

(1, 'Emely', 'emely1104pm@gmail.com', '13579', datetime.date(2003, 4, 11))

(2, 'Ingrid', 'ingridmloor86@gmail.com', '24680', datetime.date(1986, 1, 20))

(3, 'Enrique', 'galopsolorzano79@gmail.com', '12358', datetime.date(1979, 7, 19))

(4, 'Geovanny', 'peraltageovanny7@gmail.com', '837478', datetime.date(2012, 8, 20))

(5, 'Teresita', 'teresitaloor1986@gmail.com', '461876', datetime.date(1997, 2, 4))

(6, 'Nathaly', 'peraltanathaly77@gmail.com', 'mdh682b', datetime.date(2007, 5, 26))

(7, 'Luis', 'mendozaluis73@gmail.com', '32784h783', datetime.date(2004, 5, 17))

(8, 'Mirley', 'mirelymendoza7@gmail.com', '9876n27', datetime.date(2007, 7, 7))

(9, 'Iris', 'ristak753idk@gmail.com', 'pdt#heni98', datetime.date(2000, 7, 1))

(10, 'Shady', 'shaidyyygirl@gmail.com', 'DaKa#37idc', datetime.date(2008, 11, 28))

(1003, 'Keyla', 'keynotarelive87@gmail.com', 'jwfhe8375n', datetime.date(1977, 5, 7))

Elige una opción: 4

¿Qué deseas actualizar?

1. Usuario

2. Contenido

Elige: 1

(1, 'Emely')

(2, 'Ingrid')

(3, 'Enrique')

(4, 'Geovanny')

(5, 'Teresita')

(6, 'Nathaly')

(7, 'Luis')

(8, 'Mirley')

(9, 'Iris')

(10, 'Shady')

(1003, 'Keyla')

ID del usuario a actualizar: 1003

Nuevo nombre: kira

Nuevo correo: kiraai72836@gmail.com

✔ Usuario actualizado!

(1, 'Emely', 'emely1104pm@gmail.com', '13579', datetime.date(2003, 4, 11))

(2, 'Ingrid', 'ingridmloor86@gmail.com', '24680', datetime.date(1986, 1, 20))

(3, 'Enrique', 'galopsolorzano79@gmail.com', '12358', datetime.date(1979, 7, 19))

(4, 'Geovanny', 'peraltageovanny7@gmail.com', '837478', datetime.date(2012, 8, 20))

(5, 'Teresita', 'teresitaloor1986@gmail.com', '461876', datetime.date(1997, 2, 4))

(6, 'Nathaly', 'peraltanathaly77@gmail.com', 'mdh682b', datetime.date(2007, 5, 26))

(7, 'Luis', 'mendozaluis73@gmail.com', '32784h783', datetime.date(2004, 5, 17))

(8, 'Mirley', 'mirelymendoza7@gmail.com', '9876n27', datetime.date(2007, 7, 7))

(9, 'Iris', 'ristak753idk@gmail.com', 'pdt#heni98', datetime.date(2000, 7, 1))

(10, 'Shady', 'shaidyyygirl@gmail.com', 'DaKa#37idc', datetime.date(2008, 11, 28))

(1003, 'kira', 'kiraai72836@gmail.com', 'jwfhe8375n', datetime.date(1977, 5, 7))

Elige una opción: 3

¿Qué deseas eliminar?

1. Usuario

2. Contenido

Elige: 1

(1, 'Emely')

(2, 'Ingrid')

(3, 'Enrique')

(4, 'Geovanny')

(5, 'Teresita')

(6, 'Nathaly')

(7, 'Luis')

(8, 'Mirley')

(9, 'Iris')

(10, 'Shady')

(1003, 'kira')

ID del usuario a eliminar: 1003

¿Seguro que deseas eliminar el usuario 1003? (s/n): s

✔ Usuario eliminado!

Elige una opción: 5

===== CONSULTAS AVANZADAS =====

1. Top 3 usuarios con más contenido en favoritos:

(('Ingrid', 1)

(('Geovanny', 1)

(('Enrique', 1)

2. Contenido más visto (por historial):

(('Art Attack', 2)

(('La HISTORIA del COMUNISMO en 12 minutos | Resumen fácil y divertido', 2)

(('LÍMITES – Clase Completa desde Cero', 2)

(('Soundtrack to a Coup d'Etat', 1)

(('Aventura con los Kratt', 1)

(('Jorge el Curioso', 1)

(('¿Cómo funciona un circuito? (Componentes electrónicos básicos)', 1)

3. Visualizaciones en enero y mayo:

(('Emely', 'La HISTORIA del COMUNISMO en 12 minutos | Resumen fácil y divertido', datetime.datetime(2024, 1, 5, 10, 0))

(('Geovanny', 'Aventura con los Kratt', datetime.datetime(2024, 1, 20, 18, 0))

(('Nathaly', 'La HISTORIA del COMUNISMO en 12 minutos | Resumen fácil y divertido', datetime.datetime(2024, 5, 10, 21, 0))

4. Consulta libre – Usuarios con suscripción activa y su plan:

(('Ingrid', 'Duo', datetime.date(2024, 2, 1), datetime.date(2025, 2, 1))

(('Mirley', 'Duo', datetime.date(2024, 5, 1), datetime.date(2025, 5, 1))

(('Enrique', 'Familiar', datetime.date(2024, 3, 1), datetime.date(2025, 3, 1))

(('Nathaly', 'Familiar', datetime.date(2024, 4, 1), datetime.date(2025, 4, 1))

(('Iris', 'Familiar', datetime.date(2024, 6, 1), datetime.date(2025, 6, 1))

(('Emely', 'Individual', datetime.date(2024, 1, 1), datetime.date(2025, 1, 1))

(('Geovanny', 'Individual', datetime.date(2024, 1, 15), datetime.date(2025, 1, 15))

(('Shady', 'Individual', datetime.date(2024, 3, 15), datetime.date(2025, 3, 15))

Elige una opción: 3

¿Qué deseas eliminar?

1. Usuario

2. Contenido

Elige: 1

(1, 'Emely')

(2, 'Ingrid')

(3, 'Enrique')

(4, 'Geovanny')

(5, 'Teresita')

(6, 'Nathaly')

(7, 'Luis')

(8, 'Mirley')

(9, 'Iris')

(10, 'Shady')

ID del usuario a eliminar: 2

¿Seguro que deseas eliminar el usuario 2? (s/n): s

✗ No se puede eliminar: tiene registros relacionados.

5. AUTOEVALUACIÓN

Con este proyecto adquirí conocimientos básicos de BDD; a pesar de tener poco conocimiento, con la explicación previa de las clases logré diseñar e implementar una base de datos relacional. Aprendí a integrar una aplicación hecha en Python con SQL Server, todo esto mediante la librería pyodbc, ejecuté operaciones CRUD y varias consultas SQL directamente desde una interfaz de consola.

De la misma forma, comprendí cuál es la importancia del manejo de excepciones para mantener la integridad referencial cuando se elimina un registro relacional y la utilidad de consultas avanzadas para obtener información relevante a partir del comportamiento de los usuarios.

Finalmente, esta actividad me permitió relacionar el contenido visto en clase, como integrar modelos de bases de datos, sobre el lenguaje SQL y la programación en Python ya dentro de una aplicación funcional.

EXTRA: link del GitHub.

<https://github.com/Nath-user007/STREAMING-EDUCATIVO-SQL-SERVER-Y-PYTHON.git>

REFERENCIAS

Microsoft. (2024). *Microsoft SQL Server Documentation*. <https://learn.microsoft.com/sql/>

Microsoft. (2024). *ODBC Driver for SQL Server*.

<https://learn.microsoft.com/sql/connect/odbc/>

Python Software Foundation. (2025). *Python Documentation*. <https://docs.python.org/3/>

pyodbc Developers. (2024). *pyodbc Documentation*.

<https://github.com/mkleehammer/pyodbc>